Programming Project: Snake Game Using a Linked List in Java
Course: Data Structures and Algorithms
Programming Language: Java
Project Type: Individual/Group

1. Problem Description
You are required to develop a simple Snake Game using Java. The main purpose of this project is to understand how a Linked List can be used in a real-world application.
The snake consists of several connected segments. Each segment may be represented by a node in a singly linked list. When the snake moves, a new node is inserted at the beginning of the linked list, and the last node is deleted. When the snake eats food, its length increases.
The game may use a simple graphical window created with Java Swing. You may also use the console to compile and run your Java program.

2. Java Packages and Classes
You must use standard Java libraries only. No external libraries are allowed.
Import the following packages and classes:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.Random;
```

These libraries will be used for creating the game window, drawing the snake and food, reading keyboard input, and generating random food positions.
You must create a Java file named SnakeGame.java. Your program may contain a Node class and a SnakeGame class.

3. Linked List Requirements
You must implement your own Linked List. You are not allowed to use Java's built-in LinkedList, ArrayList, or arrays to store the snake's body.
Each node may contain:

- row: Row position of the snake segment.
- col: Column position of the snake segment.
- LINK: Reference to the next node.

You may use the following node structure:

```
class Node {
   int row;
   int col;
   Node LINK;

   Node(int row, int col) {
      this.row = row;
      this.col = col;
      this.LINK = null;
```

```
    }
}
```

Use START to reference the first node (head) of the linked list and PTR when traversing the linked list.

Your program may implement these methods:

Method	Required task
insertHead()	Insert a new node at the beginning of the linked list
deleteTail()	Delete the last node
snakeContains()	Check whether a position is occupied by the snake
length()	Count the total number of nodes
generateFood()	Generate food at a random empty position
resetSnake()	Restart the snake with three nodes
moveSnake()	Update the snake's position

4. Game Board and Appearance

Create a square game board with 12 rows and 12 columns, including the boundaries.

Use the following colors:

Game element	Color
Background	Black
Snake head	Bright green
Snake body	Dark green
Food	Red
Boundary walls	Gray

The initial snake may contain three nodes at positions (5,5), (5,4), and (5,3). The head starts at (5,5).

The food must appear randomly inside the board and must not overlap the snake or boundary walls.

Display the current score, snake length, and keyboard instructions below the game board.

5. Keyboard Controls

The player may be able to control the snake using the following keys:

Key	Action
W	Move up
S	Move down
A	Move left
D	Move right
Q	Quit the game

Important: The player must not need to press Enter after pressing a movement key. Each valid keypress must move the snake exactly one square.
The snake must not move automatically. It moves only when the player presses a direction key.

6. Game Rules and Conditions

Condition 1: Normal movement

When the player presses a movement key, insert a new node at the beginning of the linked list and delete the last node. This keeps the snake's length unchanged.

Condition 2: Eating food

When the snake's head reaches its food position, insert a new head but do not delete the tail. The snake's length must increase by one node. Add 10 points to the score and generate new food at a different empty position.

Condition 3: Hitting the boundary

If the snake hits any boundary wall, the game must automatically restart. Reset the snake to its original three nodes, reset the score to zero, and generate new food. Display the message:
Wall collision! Game restarted.

Condition 4: Hitting its own body

If the snake's head collides with its body, the game must automatically restart. Reset the snake's length and score, generate new food, and display:
Body collision! Game restarted.
When checking body collisions, moving into the current tail position is allowed if the snake is not eating food, because the tail moves away during that turn.

Condition 5: Continuing the game

The game must continue without a fixed number of moves. Hitting a wall or the snake's body must not close the program. The player must be able to continue playing after each automatic restart.

Condition 6: Quitting the game

The game must close when the player presses Q. The player must also be able to close the game using the window's close button.

Condition 7: Filling the board

If the snake occupies all 100 playable cells inside the boundary, display a winning message and automatically start a new game.

7. Program Execution

Your program must be saved as SnakeGame.java.
Open a terminal or command prompt in the folder containing the file.

Compile the program:
javac SnakeGame.java
Run the program:
java SnakeGame

A Swing game window must open. Use W/A/S/D to move the snake.

The console is used to compile and launch the program; the game itself must appear in the Swing window.

8. Testing Requirements

Before submitting your project, test all of the following:

- The game starts with three snake segments.
- All four movement directions work without pressing Enter.
- The snake moves one square for each keypress.
- The snake's length remains unchanged during normal movement.
- The snake grows by one node after eating food.
- The score increases by 10 after eating food.
- The game restarts automatically after hitting a wall or its own body.
- The game continues until the player presses Q or closes the window.

9. Submission Requirements

Submit your project with the following files:

1. **Java source code: SnakeGame.java, containing your complete program.**
2. **Video Recording**
3. **Short report (PDF): Explain how the singly linked list is used in your game, particularly insertion at the beginning, deletion at the end, and snake growth.**

Your program must be compiled and run successfully. You must use your own singly linked list implementation. Using a built-in linked list or another data structure to represent the snake will not satisfy the project requirement.

Failure to follow the project instructions may result in grade deductions.

****Video Recording Requirement:** You must submit a screen recording demonstrating your complete program. Start the recording by showing how you compile and run the Java program using the command prompt or terminal. Then demonstrate the game by playing it, including snake movement, eating food, scoring, wall collisions, automatic restarting, and quitting the game.

You must also **explain your source code clearly**, including the purpose of each imported package, class, method, important statement, and the linked list operations used in the game. Explain how your program works step by step.

Important: Simply running the program without explaining the code is not sufficient. Incomplete, unclear, or poor explanations will result in point deductions.

Note: Please review the sample output of the Snake Game and the sample video demonstration from another project for reference.